



Apellido y Nombres	Legajo	Profesor	# Hojas

Parte teórica

1. ¿Qué tamaño ocupa en memoria?

```
union info1{
    float vector[3];
    char str[10];
    int *p;
};
```

En una máquina con enteros de 32bits y punteros de 64bits

2. ¿Cuales son las características más distintivas de una cola circular compacta? Describa las ventajas y desventajas frente a una lista simplemente enlazada.

3. Dado el siguiente programa:

```
#include<stdio.h>
#include<string.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>

pthread_t tid[2]; int counter;

void *doSomething(void *arg) {
    unsigned long i = 0;
    counter += 1;
    printf("\n Job %d started\n", counter);
    for(i=0; i<1000; i++) {
        usleep(1000);
    };
    printf("\n Job %d finished\n", counter);
    return NULL;
}

int main(void) {
    int i = 0;
    int err;
    while(i < 2) {
        err = pthread_create(&(tid[i]), NULL, &doSomething, NULL);
        if (err != 0)
            printf("\ncan't create thread :[%s]", strerror(err));
        i++;
    }
    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);
    return 0;
}
```



Apellido y Nombres	Legajo	Profesor	# Hojas

Responda a las siguientes preguntas:

- ¿Es seguro utilizar la función `printf()` dentro del thread?
- ¿Cuál es el resultado del programa?
- ¿Propondría algún cambio a este programa?

Parte práctica

1. Escribir un programa que tome por línea de comando la cantidad de enteros que va a pedirle al usuario. El programa deberá reservar la memoria necesaria para guardarlos y luego pedirle al usuario que ingrese estos enteros de a uno. Una vez finalizado el ingreso el programa calculará el valor medio de los enteros ingresados y lo presentará por pantalla.

2. Realizar solo la función de inserción en una lista enlazada simple de forma ordenada. El formato del nodo será:

```
struct nodo {  
    int dato;  
    struct nodo *sig;  
};
```

3. Escribir un programa que tome un nombre de archivo de la línea de comandos, lo abra y recorra buscando un patrón que también se pasa por línea de comandos. El programa deberá devolver la cantidad de veces que se encuentra ese patrón en el archivo.

```
miprogram archivo.txt patron
```

4. Crear un programa cliente TCP que se conecta a un server utilizando las funciones de librería de TCP/IP de la cátedra que se encuentran al final de este examen.

Este programa cliente deberá primero recibir por línea de comandos la IP, el puerto y 2 números separados por un guión. El primero será el número de comienzo y el segundo, el número final.

Por ejemplo:

```
cliente 10.10.1.5 3255 015-025
```

Donde el primer valor es la IP "10.10.1.5", el segundo el puerto "3255" y el último está compuesto por un primer valor "015" y un segundo valor "025". Parte del trabajo del programa será realizar la separación de "015-025" en los dos números anteriormente descriptos.

Una vez conectado el cliente espera recibir un mensaje con el contenido "READY". Una vez recibido este mensaje, el programa cliente deberá enviar el primer comando: "DATnnn" donde nnn es el primer número pasado por línea de comandos (del ejemplo "DAT015").



Apellido y Nombres	Legajo	Profesor	# Hojas

A continuación se deberá preparar para recibir una cadena de 256 caracteres como máximo.

Al terminar de recibir la cadena, deberá confirmar la recepción enviando el comando "ACK" al server.

Este proceso se repite pidiendo de un dato por vez: Primero DAT015, luego DAT016, DAT017 y continua la progresión hasta llegar a pedir el dato DAT025. Siempre repitiendo el proceso, luego de enviar cada comando, se prepara para recibir la cadena de 256 caracteres.

Función de librería de TCP/IP de la cátedra (socklib.h)

```
#define PORT 3490      /* El puerto donde se conectará, servidor */
#define BACKLOG 10     /* Tamaño de la cola de conexiones recibidas */
```

```
/*
Función cliente que se conecta a un servidor remoto usando TCP
```

- Toma: Los parámetros pasados por línea de comandos
Por ejemplo: micliente 127.0.0.1 3490
- Devuelve: El socket creado listo para ser usado (si es !=0)

```
*/
```

```
int conectar (int argc, char **argv);
```

```
/* Crea un socket servidor y lo devuelve
- Toma: La estructura con los datos del socket a configurar ya armada
- Devuelve: El socket creado para aceptar conexiones (si es != 0)
```

```
*/
```

```
int Open_conexion (struct sockaddr_in *);
```

```
/* Función que acepta una conexión entrante (bloqueante)
```

- Toma: El socket creado por conectar()
- Devuelve: Un socket ya conectado a un cliente.

```
*/
```

```
int Aceptar_pedidos (int socket);
```

NOTA: Tenga en cuenta que la correcta interpretación del enunciado forma parte de la evaluación.